



Software build automation tools

Lab – Module 3

- Autor: dr inż. Aleksander Rewak
- Wrocław, 2026

Content in 3 Modules



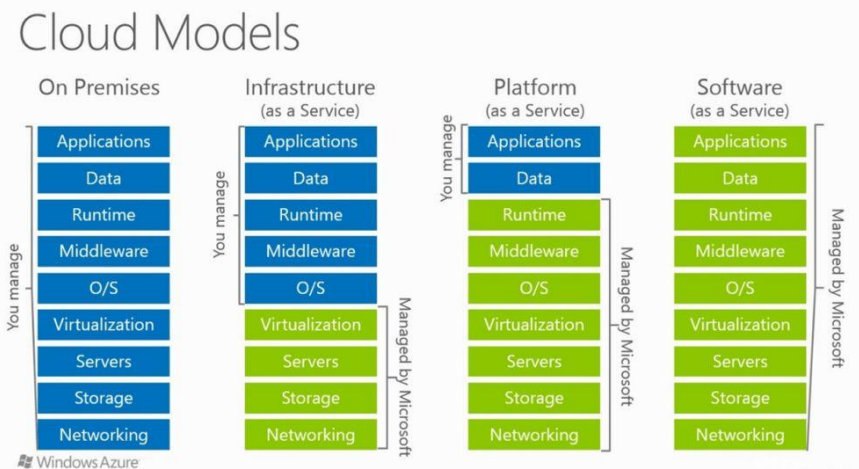
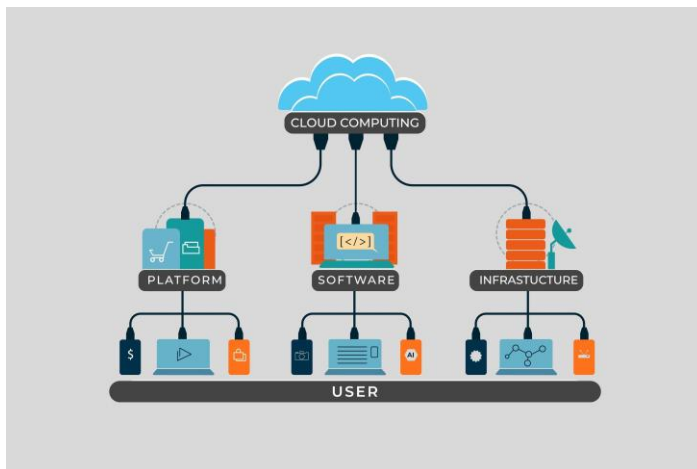
- Module 1: The "Personal" Workflow (Meetings 1-4)
- Module 2: The "Cloud" Workflow and Automation (Meetings 5-8)
- Module 3: The End-to-End Software-Based Automation with Azure Cloud (Meetings 9-12)

The Role of Microsoft Azure



Microsoft Azure is a **Cloud Computing Platform** that replaces traditional physical servers with a global network of managed data centers.

- **Infrastructure as a Service (IaaS):** Offers raw "virtual" hardware (VMs, storage, networks) where you manage everything from the OS up.
- **Platform as a Service (PaaS):** The core of our lab. Azure manages the hardware, OS, and runtime (Python), while you focus **only on your code**.
- **SaaS (Software):** Ready-to-use apps hosted by Azure (e.g., Office 365, Teams). You are the user, not the builder.



The Role of Azure App Service (Web Apps)

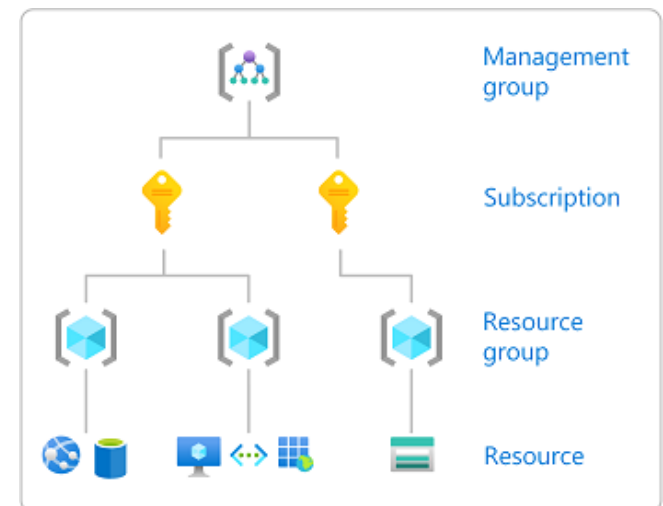


Azure App Service is a **fully managed web hosting service**. It is designed specifically to host web applications, REST APIs, and mobile backends.

- **Multilingual Support:** It natively understands **Python, Node.js, .NET, and Java**. It knows how to run your app.py without you installing a web server manually.
- **Built-in Security:** It handles SSL/TLS certificates (HTTPS) and integrates with **Managed Identities** for passwordless authentication.
- **Continuous Deployment (CD) Hub:** The **Deployment Center** acts as the "Bridge." It monitors your GitHub repository for changes and triggers the build process.
- **Zero-Downtime Updates:** When you push new code via Git, App Service can swap the old version for the new one seamlessly.

SBA Insight: The Web App doesn't just store your files; it actively runs the Gunicorn server and Flask logic defined in your requirements.txt

In Azure, a **Subscription** acts as the single billing and access container that holds multiple **Resource Groups**, which serve as logical folders to organize and lifecycle-manage your individual **Resources** (such as Web Apps, Databases, and Storage).



Azure Web App - Phase 1: Environment Setup



1. Azure Student account creation:

- **Navigate to :** <https://azure.microsoft.com/en-us/free/students/> (in the web browser type : Azure Student) and click **"Start free"**
- Use your WSB Merito University email. In case of problems with Microsoft account (e.g. Authenticator does not respond) please contact admin365@wroclaw.merito.pl.

2. Create the Web App Resource

- Search for Resource: In the top search bar, type „App Services" and select it from the results.
- Initiate Creation: Click the "+ Create,, (Web App) button to start the deployment wizard.
- Project Details:
 - Subscription: Select your active subscription (e.g., "Azure for Students").
 - Resource Group: Click "Create new" and name it (e.g., SBA-Project-RG).
 - Name: Give your app a unique name (e.g., geometry-calc-sba).
 - Publish stack: select „Code" and Python 3.12
 - Operating system: Linux
 - Region: Choose the region allowed by subscription (**UK South**).
["norwayeast", "spaincentral", "uksouth", "italynorth", "swedencentral"]
 - Pricing Plan Type: Select Free F1
- Click: Review + create

Azure Web App - Phase 2: Local Development (VS Code)



3. Create your repository on GitHub (public)
4. Create your project in VS Code
 - Create folder, app.py and index.html with code
 - Create requirements.txt with:

```
flask  
unicorn
```

5. Git (initiate)

```
git init  
git add .  
git commit -m "Prepare for Azure Web App deployment"
```

6. GitHub – connect repository

```
git remote add origin  
https://github.com/MeritoRewak/SBA\_Azure.git  
git branch -M main  
git push -u origin main
```

#app.py

```
from flask import Flask, send_from_directory  
import os  
app = Flask(__name__, static_folder='.')  
@app.route('/')  
def index():  
    return send_from_directory('.', 'index.html')  
if __name__ == '__main__':  
    app.run()
```



Index.html

In a web deployment, Flask is the application framework that contains your Python logic and routes, while Gunicorn is the production-grade WSGI HTTP server that sits in front of it to handle multiple concurrent requests and manage the application process efficiently.

Azure Web App - Phase 3: The "Bridge" (Deployment Center)



7. Deployment (The GitHub-Azure Bridge)

- Go to Deployment Tab (or later Deployment Center on the left menu):
 - GitHub Actions settings: Toggle to Enable.
 - GitHub Account: Click to authorize and link your account.
 - Organization/Repository: Select your geometry repository.
 - Branch: Select main.
- Azure automatically injects a .yaml file into your GitHub repository under .github/workflows/.

8. Verification of the CD Pipeline

- Deployment Flow: After clicking "Create,, (or save), go to your GitHub Actions tab.
- Workflow Monitoring: Watch the "Build and Deploy" job. It will install dependencies and sync code to the Sweden data center.
- Live URL: Once the GitHub Action turns green, find your URL in the Azure Overview tab (e.g., <https://geometrymer.azurewebsites.net>).

9. VS Code sync - git pull origin main

Azure Web App - Monitoring the Pipeline



10. Monitoring

- Go to your GitHub Actions tab.
- Watch the job: It will log into Azure, install dependencies, and deploy the code.
- If it fails: Check the "Login" step. Usually, it's an issue with the Identity permissions (RBAC).
- Once the Action turns Green, find your URL in the Azure Overview tab (Default domain)
- SBA Success: Any future git push from VS Code will now automatically update the live website within 2 minutes.

This workflow demonstrates Infrastructure as Code (IaC) principles. We are not "uploading files"; we are managing a Continuous Deployment pipeline. The "Bridge" we built between GitHub and Azure ensures that the code moves safely and automatically from the developer's laptop to the global cloud.

GitHub Actions workflow: 1. The Triggers and The Runner



- **Triggers (on:):**
 - **push to main:** The automation starts automatically every time code is committed to the primary branch.
 - **workflow_dispatch:** Allows you to manually trigger the process from the GitHub "Actions" tab—useful for testing without changing code.
- **The Environment (runs-on):**
 - The workflow uses **ubuntu-latest**. This means GitHub spins up a fresh, clean Linux Virtual Machine (VM) in the cloud specifically to run your tasks.
- **Permissions:**
 - The workflow is explicitly granted permission to **read** the contents of your repository so it can access your Python files.

GitHub Actions workflow: 2. The Build Job



"Preparing the Application,,

- **Setup** (`actions/checkout` & `setup-python`):
 - The VM "checks out" (downloads) your code and installs **Python 3.12**. This ensures the build environment exactly matches your development environment.
- **Dependency Validation:**
 - The script creates a virtual environment (`antenv`) and runs `pip install`. This step acts as a **smoke test**: if a library is missing from `requirements.txt`, the build fails here before it ever touches Azure.
- **Artifact Creation** (`upload-artifact`):
 - Once the code is ready, GitHub "zips" it up as an **artifact** named `python-app`.
 - **Crucial Logic:** We exclude the `antenv/` folder. Why? Because Azure uses a tool called **Oryx** to build its own environment on the server side, keeping the upload small and fast.

GitHub Actions workflow: 2. The Build Job



"Moving to the Cloud"

•**Dependency** (`needs: build`):

- The deployment job **cannot start** until the build job finishes successfully. This prevents broken code from being deployed.

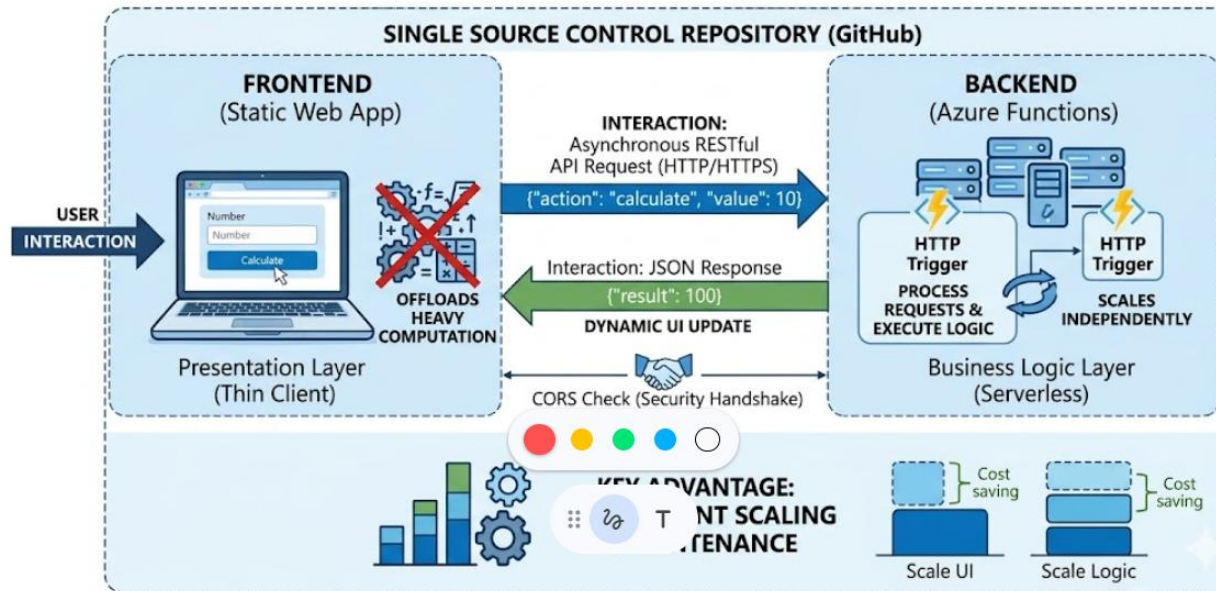
•**Secure Authentication** (`azure/login`):

- The workflow uses **OIDC (OpenID Connect)**. Instead of storing a permanent password, it uses a **Client ID** and **Tenant ID** to request a temporary, secure token from Azure.

•**The Handover** (`webapps-deploy`):

- This is the final step. GitHub sends the artifact to the Azure App Service named **"Geometry"**.
- Once finished, the application is live in the **Production** slot, accessible via your `.azurewebsites.net` URL

Frontend & Backend architecture



Key Advantage: This decoupling allows for independent scaling and maintenance. You can update the UI without redeploying the backend, and you can scale the computation logic (Azure Functions) independently based on traffic demand, optimizing costs and performance.

Frontend & Backend architecture



Core Concept: The **Decoupled Monorepo** architecture separates the presentation layer (Frontend) from the business logic layer (Backend), while maintaining both within a single source control repository.

- **Frontend (Static Web App/App Service):** Acts as a thin client. Its primary responsibility is the user interface and handling user interactions. It offloads all heavy computation and data processing to external services.
- **Backend (Azure Functions):** A serverless, event-driven compute service. It exposes specific API endpoints (e.g., HTTP Triggers) that process requests, execute logic, and return results (typically in JSON format).
- **Interaction:** The two layers communicate asynchronously via **RESTful** (Representational State Transfer) **APIs** over HTTP/HTTPS. By exchanging structured JSON data, the frontend requests specific computations from the backend and dynamically updates the user interface upon receiving the calculated results. Because the frontend and backend are hosted on different domains, the browser performs a **CORS (Cross-Origin Resource Sharing)** check. This is a security "handshake".

Frontend & Backend architecture



Step 1. Repository Setup (Version 1)

Ensure your base web application is ready in the root directory of your repository.

- Open the terminal in VS Code.
- Initialize your repository and push the initial monolith version:

```
git add .  
git commit -m "Version 1: Monolith frontend"  
git push origin main  
git tag -a v1 -m "Version 1"  
git push origin v1
```

Frontend & Backend architecture



STEP 2: Create Azure Function Resource in VS Code

1. Install Azure Functions in VS Code (Extensions on the left side)
2. In VS Code, click the **Azure icon** in the left sidebar. Under "Resources," click **Sign in to Azure** and authenticate via your browser. Ensure your subscription is visible.
3. Click the **Azure** icon in the left Activity Bar.
4. Under **Resources**, click the **+ (Create Resource)** icon.
5. Select **Create Function App in Azure (Advanced)**.
6. Follow the prompts:
 - **Name:** Enter a unique name (e.g., SBACalculateArea).
 - **Runtime:** Python 3.12.
 - **Plan:** Consumption.
7. Wait for the notification that the resource has been created.

Frontend & Backend architecture



STEP 3: Create in VS Code API folder and logic

1. Create a new folder named api in your project root.
2. In VS Code, press Ctrl+Shift+P and select **Azure Functions: Create New Project**.
3. Select api folder.
4. Select: **Python** -> **V2 model** -> **HTTP Trigger** -> Name: CalculateArea.
5. Open api/function_app.py. Replace the content with the logic to calculate the area of various shapes (code enclosed).

Frontend & Backend architecture



STEP 4: Deployment and CI/CD Pipeline

This step automates all future updates to both your Web App (frontend) and Function App (backend).

1. **Manual Deploy:** In VS Code, right-click your Function App -> **Deploy to Function App** -> Select the api folder.
2. **Setup Automation:**
 1. Log into the [Azure Portal](#).
 2. Navigate to your Function App -> **Deployment Center**.
 3. Select **Source: GitHub** and authorize your account.
 4. Choose your repository and main branch. Click **Save**. Azure will push a .yml file to your repo.
3. **Configure CI/CD Path:**
 1. Run git pull origin main in VS Code to retrieve the new workflow file.
 2. Open the file in .github/workflows/
 3. CRITICAL: Update the env section to set
AZURE_FUNCTIONAPP_PACKAGE_PATH: './api'.

Frontend & Backend architecture



STEP 5: Finalizing Version 2

1. Push your changes:
git add .
git commit -m "Version 2: API integration and CI/CD configured,,
git push origin main
2. Configure CORS in the Azure Portal for your Function App:
 - Go to API -> CORS.
 - Add your Web App URL (e.g., <https://geometry-xyz.azurewebsites.net>) to the Allowed Origins list.
3. Finalize the release:
git tag -a v2 -m "Version 2,,
git push origin v2

Frontend & Backend architecture



STEP 5: Rolling Back to Version 1 (Rollback Procedure)

1. Checkout Version 1: Reset your local project to the state of the v1 tag:

```
git checkout v1
```

2. Force Update the Main Branch: To make v1 the current active state on GitHub, reset the main branch to the state of the tag and force push:

```
git checkout main
```

```
git reset --hard v1
```

```
git push origin main --force
```

Students Project 3



Project Title: Cloud Web Application Deployment

- **Objective:** Design, implement, and automate the deployment of a distributed web application using Microsoft Azure and GitHub Actions.
- **The Task:** Transition your existing Module 2 project into a cloud-native architecture or create a new.
- **Core Requirements:**
 - **Architecture:** Split your project into a Frontend (Azure Web App) and a Backend (Azure Function - Python V2).
 - **Interoperability:** Ensure the frontend communicates with the backend via fetch requests with configured **CORS** policies.
 - **Automation:** Implement **CI/CD** using GitHub Actions—every git push to main must trigger an automatic build and deployment.
 - **Versioning:** Use Git Tags (e.g., v1.0) to mark your final submission.

Students Project 3



Deliverables (Upload into Moodle):

- 1. Documentation:** A Word or pdf document including:
 - A brief description of your application's functionality.
 - Screen shot of:
 - a. Web App Overview page in Azure,
 - b. Azure Function Overview page in Azure,
 - c. GitHub workflows (Actions)
 - Link to GitHub repository
- 2. VS Code Project folder (zip)**

Evaluation Checklist (Pass/Fail):

- **Cloud Hosted:** Both frontend and backend are live in Azure
- Application works
- **CI/CD Operational:** Automated deployment pipeline is verified through GitHub Actions.